Two Pointer Problem Documentation (3rd June)

This document outlines five classic array problems and compares **brute force** versus **two-pointer (optimal)** approaches for each.

1. Remove Duplicates from Sorted Array

Problem:
Remove duplicates from a sorted array `nums` in-place such that each unique element appears only once. Return the number of unique elements.

Brute Force Approach:

- Use a `set` to collect unique elements.
- Sort the result and copy it back to `nums`.
- Not in-place; extra space used.
- **Time:** `O(n log n)`

Two Pointer Approach (Optimal):

- Initialize `i = 0`, iterate `j` from 1 to `n - 1`.
- If `nums[j] != nums[i]`, increment `i` and set `nums[i] = nums[j]`.
- Return `i + 1` as the count of unique elements.
- **Time:** `O(n)`, **Space:** `O(1)`

2. Squares of a Sorted Array

Problem:
Given a non-decreasing sorted array `nums`, return a new array of squares sorted in non-decreasing order.

Brute Force Approach:

- Square every element.
- Sort the array.
- **Time:** `O(n log n)`

Two Pointer Approach (Optimal):

- Use two pointers: `left = 0`, `right = n - 1`, `pos = n - 1`.
- Compare absolute values, fill result array from end.
- **Time:** `O(n)`, **Space:** `O(n)`

3. Three Sum

Problem:
Return all unique triplets `(i, j, k)` such that `nums[i] + nums[j] + nums[k] == 0`.

Brute Force Approach:

- 3 nested loops to try every triplet.
- Check sum and manage duplicates.
- **Time:** `O(n^3)`

Two Pointer Approach (Optimal):

- Sort array.
- Fix one number, then use two pointers for the other two.
- Skip duplicates.
- **Time:** `O(n^2)`, **Space:** `O(1)`

4. Three Sum Closest

Problem:
Find three numbers in `nums` such that their sum is closest to `target`.

Brute Force Approach:

- 3 nested loops, track minimum difference.
- **Time:** `O(n^3)`

Two Pointer Approach (Optimal):

- Sort array.
- For each `i`, use two pointers `left`, `right`.
- Move pointers based on sum vs target.
- Track closest sum.
- **Time:** `O(n^2)`

5. Two Sum (Sorted Input)

Problem:
Given a sorted array, find two numbers that add up to the target.

Brute Force Approach:

- Use nested loop to find pair.
- **Time:** `O(n^2)`

Two Pointer Approach (Optimal):

- Initialize `left = 0`, `right = n - 1`.
- If sum < target, move left++; if sum > target, move right--.
- Return indices or values.

- **Time:** `O(n)`

Each problem demonstrates the efficiency of the two-pointer technique in reducing time complexity from quadratic/cubic to linear/quadratic where applicable.

Code Samples

1. Remove Duplicates from Sorted Array

Brute Force:

```
int removeDuplicates(vector<int>& nums) {
    set<int> s(nums.begin(), nums.end());
    int k = 0;
    for (int num : s) {
        nums[k++] = num;
    }
    return k;
}
```

Two Pointer Approach:

```
int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;
    int i = 0;
    for (int j = 1; j < nums.size(); j++) {
        if (nums[j] != nums[i]) {
            i++;
            nums[i] = nums[j];
        }
    }
    return i + 1;
}
```

2. Squares of a Sorted Array

Brute Force:

```
vector<int> sortedSquares(vector<int>& nums) {
    for (int& num : nums) num *= num;
    sort(nums.begin(), nums.end());
    return nums;
}
```

Two Pointer Approach:

```
vector<int> sortedSquares(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n);
    int left = 0, right = n - 1, pos = n - 1;

    while (left <= right) {
        int leftSq = nums[left] * nums[left];
        int rightSq = nums[right] * nums[right];

        if (leftSq > rightSq) {
            result[pos--] = leftSq;
            left++;
        } else {
            result[pos--] = rightSq;
            right--;
```

```
        }
    }
    return result;
}
```

3. Three Sum

Brute Force:

```
vector<vector<int>> threeSum(vector<int>& nums) {
    int n = nums.size();
    set<vector<int>> s;
    for (int i = 0; i < n - 2; i++) {
        for (int j = i + 1; j < n - 1; j++) {
            for (int k = j + 1; k < n; k++) {
                if (nums[i] + nums[j] + nums[k] == 0) {
                    vector<int> triplet = {nums[i], nums[j], nums[k]};
                    sort(triplet.begin(), triplet.end());
                    s.insert(triplet);
                }
            }
        }
    }
    return vector<vector<int>>(s.begin(), s.end());
}
```

Two Pointer Approach:

```
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    int n = nums.size();
    vector<vector<int>> result;

    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int left = i + 1, right = n - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.push_back({nums[i], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++; right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }
    return result;
}
```

4. Three Sum Closest

Brute Force:

```
int threeSumClosest(vector<int>& nums, int target) {
    int n = nums.size(), minDist = INT_MAX, closestSum = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
```

```
            for (int k = j + 1; k < n; k++) {
                int sum = nums[i] + nums[j] + nums[k];
                if (abs(target - sum) < minDist) {
                    minDist = abs(target - sum);
                    closestSum = sum;
                }
            }
        }
    }
    return closestSum;
}
```

Two Pointer Approach:

```
int threeSumClosest(vector<int>& nums, int target) {
    sort(nums.begin(), nums.end());
    int n = nums.size(), closestSum = nums[0] + nums[1] + nums[2];

    for (int i = 0; i < n - 2; i++) {
        int left = i + 1, right = n - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (abs(target - sum) < abs(target - closestSum)) {
                closestSum = sum;
            }
            if (sum < target) left++;
            else right--;
        }
    }
    return closestSum;
}
```

5. Two Sum (Sorted Array)

Brute Force:

```
vector<int> twoSum(vector<int>& nums, int target) {
    int n = nums.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                return {i, j};
            }
        }
    }
    return {-1, -1};
}
```

Two Pointer Approach:

```
vector<int> twoSum(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return {left, right};
        else if (sum < target) left++;
        else right--;
    }
    return {-1, -1};
}
```

🔍 When to Use the Two Pointer Approach

✅ Ideal Scenarios

1. **Sorted Arrays**
   Two pointers are especially effective on sorted arrays, where pointer movement can be guided based on comparisons.
2. **Finding Pairs/Triplets**
   Problems that involve finding pairs or triplets with specific properties (e.g., sum to a target, closest to a target, etc.).
3. **In-place Modifications**
   Tasks that require modifying an array in-place while maintaining relative order or reducing space complexity.
4. **Maximize/Minimize Something**
   When you're optimizing something (like distance, sum, or difference) and can "slide" the pointers to improve it.
5. **Palindromes and Reversals**
   Comparing elements from both ends of an array or string (like checking for palindromes or reversing content).

🧠 Clues in Problem Statements

Clue	Example
"Sorted array"	Most classic two-pointer problems
"Find a pair/triplet with..."	Two Sum, Three Sum, Closest Sum
"Modify in-place"	Remove Duplicates, Move Zeroes
"Find max/min with constraint"	Container With Most Water, Closest Pair
"Without using extra space"	Favor two-pointer over hashing